DATA MINING AND KNOWLEDGE

Ανάλυση Πελατειακής Συμπεριφοράς σε Ηλεκτρονικό Κατάστημα (E-Commerce)

Παναγιώτης Καραμπογιάς ΑΜ 2122111
Στρατίκης Περικλής ΑΜ 2122033

ΕΙΣΑΓΩΓΗ

Η ανάπτυξη του ηλεκτρονικού εμπορίου τα τελευταία χρόνια έχει δημιουργήσει νέες ανάγκες για τις επιχειρήσεις που δραστηριοποιούνται online. Κάθε συναλλαγή, κάθε επίσκεψη στη σελίδα και κάθε αλληλεπίδραση του πελάτη αφήνει πίσω της δεδομένα τα οποία μπορούν να χρησιμοποιηθούν για να κατανοήσει η εταιρεία τη συμπεριφορά των χρηστών. Παρ όλα αυτά, πολλές μικρές επιχειρήσεις δεν έχουν τη γνώση ή τα εργαλεία για να αξιοποιήσουν αυτές τις πληροφορίες. Χωρίς σωστή ανάλυση, ένα ηλεκτρονικό κατάστημα λειτουργεί ουσιαστικά στα τυφλά, χωρίς να μπορεί να γνωρίζει ποιοι πελάτες ενδιαφέρονται για ποια προϊόντα, ποιοι εγκαταλείπουν τις αγορές τους ή τι θα μπορούσε να βελτιωθεί στις υπηρεσίες του. Σε αυτό το πλαίσιο, η εξόρυξη δεδομένων μπορεί να προσφέρει λύσεις που βοηθούν την επιχείρηση να κατανοήσει καλύτερα τους πελάτες της, να προσαρμόσει τις υπηρεσίες της και να αυξήσει τις πωλήσεις της.

Περιγραφή του προβλήματος

Ας υποθέσουμε πως η εταιρεία είναι ένα μικρό ηλεκτρονικό κατάστημα που πουλάει προϊόντα καθημερινής χρήσης, όπως είδη σπιτιού, μικρές ηλεκτρικές συσκευές και αξεσουάρ. Η εταιρεία λειτουργεί κυρίως μέσω του e-shop της, έχει σταθερή επισκεψιμότητα, αλλά δεν διαθέτει ξεχωριστό τμήμα πληροφορικής ή κάποιον εξειδικευμένο αναλυτή δεδομένων.Όλες οι δραστηριότητες (παραγγελίες, επισκέψεις προϊόντων, καλάθια αγορών) δημιουργούν δεδομένα, αλλά αυτά καταλήγουν απλά αποθηκευμένα χωρίς να αξιοποιούνται.Η επιχείρηση δεν έχει τρόπο να δει ποιοι πελάτες αγοράζουν συχνά,
να αναγνωρίσει ποιοί συνδυασμοί προϊόντων τείνουν να αγοράζονται μαζί,
να εντοπίσει πελάτες που εγκαταλείπουν συχνά το καλάθι,να καταλάβει ποια προϊόντα έχουν

υψηλό ενδιαφέρον αλλά χαμηλές πωλήσεις.Αυτό έχει ως αποτέλεσμα:χαμένες ευκαιρίες πωλήσεων,μη στοχευμένες διαφημίσεις,κακή διαχείριση αποθέματος,μειωμένη ικανοποίηση πελατών.

Η επιχείρηση χρειάζεται έναν τρόπο να καταλάβει τη συμπεριφορά των πελατών της,να προβλέπει τάσεις,να βελτιώνει τις προωθητικές ενέργειες,να οργανώνει καλύτερα τα προϊόντα της.Μια λύση εξόρυξης δεδομένων μπορεί να τη βοηθήσει να πάρει αποφάσεις βασισμένες σε πραγματικά δεδομένα και όχι σε υποθέσεις.

Η λύση που προτείνουμε

Η προτεινόμενη λύση προσφέρει κάτι που η επιχείρηση δεν έχει, ανάλυση δημογραφικών και αγοραστικών συνηθειών,εντοπισμό ομάδων πελατών με παρόμοια συμπεριφορά ,πρόβλεψη πιθανής αγοράς ,συσχετίσεις προϊόντων για στοχευμένες προτάσεις.

Μεθοδολογία

Για την ανάλυση της πελατειακής συμπεριφοράς σε ηλεκτρονικό κατάστημα απαιτείται η συλλογή και οργάνωση δεδομένων που αποτυπώνουν τόσο τις ενέργειες των χρηστών όσο και τα χαρακτηριστικά των προϊόντων και των παραγγελιών. Τα απαραίτητα δεδομένα περιλαμβάνουν πληροφορίες αγοράς(προϊόντα που αγοράστηκαν,ποσό που ξοδεύτηκε, προσφορά), αναγνωριστικά χρηστών,δευτερεύοντα στοιχεία αγορών(σε τι πλάνο βρίσκεται ο κάθε πελάτης, πόσο ευχαριστημένος έμεινε). Η καταγραφή μπορεί να πραγματοποιηθεί μέσω μηχανισμών όπως event tracking σε κάθε σελίδα του καταστήματος, server logs ,εργαλεία ανάλυσης επισκεψιμότητας, ενώ η αποθήκευση γίνεται σε δομημένα αρχεία, ανάλογα με τον όγκο και τη συχνότητα των δεδομένων. Μετά τη συλλογή, τα δεδομένα οργανώνονται σε πίνακες (events, users, products, orders) ώστε να μπορούν να αναλυθούν εύκολα με τεχνικές εξόρυξης γνώσης, όπως segmentation, κανόνες συσχέτισης και μοντέλα πρόβλεψης.Τα αποτελέσματα της επεξεργασίας έπειτα υιοθετούνται από τρεις βασικούς ρόλους: ο υπεύθυνος marketing χρησιμοποιεί τη γνώση για στοχευμένες καμπάνιες και κατανόηση των προτιμήσεων των πελατών, ο ιδιοκτήτης/υπεύθυνος πωλήσεων αξιοποιεί τις πληροφορίες για βελτίωση των πωλήσεων και εντοπισμό εμποδίων στη διαδικασία αγοράς, ενώ η ομάδα περιεχομένου και UX χρησιμοποιεί τα ευρήματα για να βελτιώσει τη δομή των σελίδων και να μειώσει την εγκατάλειψη. Ο συνολικός στόχος της παροχής γνώσης είναι η λήψη αποφάσεων που αυξάνουν την αποδοτικότητα του e-shop και βελτιώνουν την εμπειρία του χρήστη.

Προεπεξεργασία Δεδομένων

Για να πετύχουμε σωστή ανάλυση της πελατειακής συμπεριφοράς ενός e-commerce συστήματος, η προεπεξεργασία των δεδομένων αποτελεί απαραίτητο βήμα πριν την εφαρμογή αλγορίθμων εξόρυξης γνώσης. Τα δεδομένα αφορούν πραγματικές αλληλεπιδράσεις πελατών με το ηλεκτρονικό κατάστημα (συμπεριφορικά, δημογραφικά και οικονομικά στοιχεία), επομένως είναι πολύ πιθανό να παρουσιάζουν θόρυβο, ελλείψεις ή λανθασμένες τιμές . Για τον λόγο αυτό, απαιτούνται συγκεκριμένες μέθοδοι προεπεξεργασίας ώστε οι αλγόριθμοι clustering, classification και association rules να λειτουργήσουν σωστά και με ακρίβεια.

1. Έλεγχος και αντιμετώπιση ελλιπών τιμών (missing values)

Επειδή τα δεδομένα προέρχονται από πραγματική χρήση (clicks, παραγγελίες, wishlist), είναι πιθανό κάποιες τιμές να λείπουν λόγω μη καταγραφής ή τεχνικών σφαλμάτων.
Αν δεν αντιμετωπιστούν, μπορούν να επηρεάσουν αρνητικά τόσο την ομαδοποίηση όσο και την ταξινόμηση.Γι Αυτό τον λόγο σιγουρευόμαστε ότι έχουν συμπληρωθεί με το median του κάθε πεδίου, πριν περάσουμε στην διαχείριση ακραίων τιμών.Υλοποιούμε τον εξής αλγόριθμο:

```
df = df.dropna(subset=["Customer_ID"])
numeric_cols = [
    "Recency","Frequency","Monetary","Avg_Order_Value",
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Campaign_Response","Wishlist_Adds",
    "Cart_Abandon_Rate","Returns"
]
for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())
df["Segment_Label"] = df["Segment_Label"].fillna("Unknown")
```

Ουσιαστικά αν έχουμε κενό το USER ID απορρίπτουμε το συγκεκριμένο πεδίο τελείως ενώ για όλα τα πεδία που είναι διαμορφωμένα για αριθμητικές τιμές και είναι κενά, τα συμπληρώνουμε με median values.Τέλος αν στο segment label που είναι πρακτικά το επίπεδο συνδρομητικής υπηρεσίας (silver, copper, platinum κλπ) βρούμε κενό πεδίο απλά το συμπληρώνουμε με “Unknown”

---

2. Εντοπισμός και διαχείριση ακραίων τιμών (outliers)

Στις μεταβλητές Frequency, Monetary, Session Count και Clicks εμφανίζονται συχνά ακραίες συμπεριφορές (π.χ. πελάτες με εξαιρετικά υψηλή δραστηριότητα).
Τα outliers μπορούν να διαστρεβλώσουν τα αποτελέσματα του clustering, επομένως χρειάζεται να εντοπιστούν και να αξιολογηθεί αν πρέπει να περιοριστούν ή να διατηρηθούν.

Για τον σκοπό αυτό χρησιμοποιήθηκε η τεχνικη interquartile range (IQR) που ειναι Τα 4-quantiles είναι τα τρία σημεία που χωρίζουν την κατανομή σε 4 ίσα τμήματα • Κάθε τμήμα απεικονίζει το ¼ της κατανομής Ονομάζονται quartiles Τα 100-quantiles ονομάζονται percentiles Η απόσταση μεταξύ του 1ου και του 3ου quartile μας δείχνει την περιοχή που καλύπτεται από μεσαίο τμήμα των δεδομένων.

Επεξήγηση Κώδικα

```
def cap_outliers(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    low = Q1 - 1.5 * IQR
    high = Q3 + 1.5 * IQR
    return series.clip(lower=low, upper=high)

for col in numeric_cols:
    df[col] = cap_outliers(df[col])
```

Υπολογισμός 1ου τεταρτημορίου

Υπολογισμός 4ου τεταρτημορίου

Inter Quartile Range

Κατώτατο αποδεκτό όριο

Ανώτατο αποδεκτό όριο

Κόβει τις ακραίες τιμές στα όρια

Γιατί χρησιμοποιούμε IQR;

Δεν επηρεάζεται από ακραίες τιμές (σε αντίθεση με mean/std).

Είναι η πιο συνηθισμένη μέθοδος σε datasets με “βαριές ουρές” όπως e-commerce.

Προστατεύει το clustering, γιατί το K-means επηρεάζεται πολύ από μεγάλες τιμές.

3. Κανονικοποίηση αριθμητικών μεταβλητών

Οι αριθμητικές μεταβλητές έχουν διαφορετικές κλίμακες μέτρησης (π.χ. Recency σε ημέρες, Monetary σε ευρώ, Clicks σε πλήθος).
Για clustering και classification, απαιτείται όλες οι μεταβλητές να βρίσκονται στην ίδια κλίμακα ώστε κανένα χαρακτηριστικό να μην κυριαρχεί στη διαδικασία.

```
df["Customer_ID"] = df["Customer_ID"].str.strip()
df["Segment_Label"] = df["Segment_Label"].str.strip().str.title()
valid_segments = ["Bronze", "Silver", "Gold", "Platinum", "Iron", "Copper"]
df["Segment_Label"] = df["Segment_Label"].apply(
lambda x: x if x in valid_segments else "Unknown”
)
```

Αρχικά βγάζουμε τα κενα απο τα πεδία Customer_ID και Segment_Label και κάνουμε κεφαλαίο το πρώτο γράμμα κάθε λέξης στο Segment_Label για να έχουμε ομοιομορφία. Έπειτα ορίζουμε τις μόνες αποδεκτές τιμές στο Segment_Label και με την lambda ελέγχουμε αν το πεδιο του Segment_Label ταιριάζει με αποδεκτή τιμή. Αν ταιριάζει, τότε δίνει την τιμή αλλιώσ δίνει “Unknown”.

4. Κωδικοποίηση κατηγορικών χαρακτηριστικών (encoding)

Ορισμένες στήλες (όπως το Segment Label ή το Campaign Response) είναι συμβολοσειρές και πρέπει να μετατραπούν σε αριθμητικές τιμές ώστε να μπορούν να χρησιμοποιηθούν από τα μοντέλα ταξινόμησης.Σε αλγορίθμους μηχανικής μάθησης, τα κατηγορικά δεδομένα (Segment Label, Campaign Response) πρέπει να μετατραπούν σε αριθμούς, γιατί τα μοντέλα δεν δέχονται κείμενο.Για την κωδικοποίηση των κατηγορικών χαρακτηριστικών χρησιμοποιήθηκε ο αλγόριθμος One-Hot Encoding για το πεδίο Segment Label, καθώς πρόκειται για ονομαστική μεταβλητή χωρίς ιεραρχία. Με αυτή τη μέθοδο κάθε κατηγορία μετατρέπεται σε ξεχωριστή δυαδική μεταβλητή (0,1), επιτρέποντας στα μοντέλα ταξινόμησης να τη χρησιμοποιήσουν χωρίς να δημιουργηθεί ψευδής σειρά μεταξύ των κατηγοριών. Το πεδίο Campaign Response δεν απαιτεί περαιτέρω encoding, καθώς είναι ήδη δυαδικό (0 = no response, 1 = response).

```
user_encoder = LabelEncoder()
df["Customer_ID_Encoded"] = user_encoder.fit_transform(df["Customer_ID"])
df = pd.get_dummies(df,
                    columns=["Segment_Label"],
                    prefix="Segment",
                    drop_first=False)
print(df.filter(like="Segment").head())
print(df[["Customer_ID", "Customer_ID_Encoded"]].head())
```

Επεξήγηση τμηματος κώδικα

Καλεί από την βιβλιοθήκη sklearn το LabelEncoder και το βάζει να δημιουργήσει έναν πίνακα Customer_ID_Encoded που είναι κωδικοί που αντιστοιχεί ένας προς έναν με τα πεδία του Customer_ID. Έπειτα δημιουργεί έναν πίνακα αυτής της διαμόρφωσης για τα Segment_Label:

	Segment_Copper	Segment_Gold	Segment_Iron	Segment_Platinum	Segment_Silver
0	False	False	False	False	True
1	False	False	True	False	False
2	True	False	False	False	False
3	False	False	False	False	True
4	False	True	False	False	False

Και αυτής τησ διαμόρφωσης για τα Customer_ID:

	Customer_ID	Customer_ID_Encoded
0	CUST00001	0
1	CUST00002	1
2	CUST00003	2
3	CUST00004	3
4	CUST00005	4

Έτσι επιτυγχάνεται η έυκολη αντιστοιχία και κατανόηση των δεδομένων από το μοντέλο που θα επιλέξουμε στην πορεία του project.

5. Έλεγχος για διπλότυπες εγγραφές (duplicates)

Σε datasets πελατών συχνά εμφανίζονται πολλές εγγραφές με το ίδιο Customer_ID, είτε λόγω επαναλαμβανόμενης καταγραφής, είτε λόγω λαθών στη συλλογή δεδομένων. Τα duplicates δημιουργούν προβλήματα στην ανάλυση (π.χ. πολλαπλασιάζουν το Frequency/Monetary για έναν πελάτη), γι' αυτό πρέπει να εντοπιστούν και να αφαιρεθούν. Στον κώδικα χρησιμοποιούμε τη μέθοδο: df.drop_duplicates(subset=["Customer_ID"]) η οποία εντοπίζει εγγραφές με το ίδιο Customer_ID και κρατάει μόνο την πρώτη εμφάνιση, διαγράφοντας τις υπόλοιπες.

Έλεγχος και αφαίρεση διπλότυπων εγγραφών

```
before = df.shape[0]
df = df.drop_duplicates(subset=["Customer_ID"])
after = df.shape[0]
print(f"\nRemoved duplicates: {before - after}")
```

Μετράμε πόσες γραμμές έχει το dataset πριν

Εντοπίζει εγγραφές με το ίδιο Customer_ID και κρατά μόνο μία

Μετράμε τις γραμμές μετά την αφαίρεση

6.Έλεγχο Εγκυρότητας Δεδομένων (Data Validation)

Ο έλεγχος εγκυρότητας δεδομένων είναι κρίσιμος, γιατί εντοπίζει τιμές που είναι μαθηματικά ή λογικά αδύνατες και επομένως αλλοιώνουν τα αποτελέσματα της ανάλυσης. Με τον εντοπισμό και τη διόρθωση τέτοιων συνεπειών εξασφαλίζουμε ότι το μοντέλο εκπαιδεύεται πάνω σε αξιόπιστα και πραγματικά δεδομένα. Έτσι αποφεύγονται παραπλανητικά patterns, λανθασμένες ταξινομήσεις και αποφάσεις.

```
# Avg_Order_Value <= Monetary

invalid = df[df["Avg_Order_Value"] > df["Monetary"]]

print("\nEntries with impossible order values:", invalid.shape[0])



# Frequency must be >= Session_Count? (λογικό: orders <= sessions)

invalid2 = df[df["Frequency"] > df["Session_Count"]]

print("Entries with Frequency > Sessions:", invalid2.shape[0])
```

Πλήρης Κωδικας Προεπεξεργασίας Δεδομένων

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder




df = pd.read_csv("C:/Users/perio/Desktop/Εξόρυξη Δεδομένων και Γνώσης/ecommerecenice.csv")


print("Initial shape:", df.shape)

print(df.head())

print(df.info())









print("\nMissing values per column:")

print(df.isna().sum())


df = df.dropna(subset=["Customer_ID"])


numeric_cols = [

    "Recency","Frequency","Monetary","Avg_Order_Value",
```

```
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Campaign_Response","Wishlist_Adds",
    "Cart_Abandon_Rate","Returns"
]

for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

df["Segment_Label"] = df["Segment_Label"].fillna("Unknown")

for col in numeric_cols:
    df = df[df[col] >= 0]

df = df[df["Avg_Session_Duration"] < 300]

def cap_outliers(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    low = Q1 - 1.5 * IQR
    high = Q3 + 1.5 * IQR
    return series.clip(lower=low, upper=high)
```

```
for col in numeric_cols:
    df[col] = cap_outliers(df[col])
df["Customer_ID"] = df["Customer_ID"].str.strip()
df["Segment_Label"] = df["Segment_Label"].str.strip().str.title()
valid_segments = ["Bronze", "Silver", "Gold", "Platinum", "Iron", "Copper"]
df["Segment_Label"] = df["Segment_Label"].apply(
    lambda x: x if x in valid_segments else "Unknown"
)
user_encoder = LabelEncoder()
df["Customer_ID_Encoded"] = user_encoder.fit_transform(df["Customer_ID"])
df = pd.get_dummies(df,
                    columns=["Segment_Label"],
                    prefix="Segment",
                    drop_first=False)
print(df.filter(like="Segment").head())
print(df[["Customer_ID", "Customer_ID_Encoded"]].head())
```

```python
before = df.shape[0]

df = df.drop_duplicates(subset=["Customer_ID"])

after = df.shape[0]

print(f"\nRemoved duplicates: {before - after}")



invalid = df[df["Avg_Order_Value"] > df["Monetary"]]

print("\nEntries with impossible order values:", invalid.shape[0])



invalid2 = df[df["Frequency"] > df["Session_Count"]]

print("Entries with Frequency > Sessions:", invalid2.shape[0])




df[numeric_cols] = df[numeric_cols].astype(float)



df.to_csv("clean_preprocessed_ecommerce.csv", index=False)



print("\n Saved as clean_preprocessed_ecommerce.csv")

print(df.head())
```

Clustering

Αρχικα θα εφαρμόσουμε τον αλγοριθμο k-means για την ομαδοποίηση στα εξης δεδομενα :

Recency Frequency Monetary Avg Order Value Session Count Avg Session Duration Pages_Viewed Clicks Wishlist Adds Cart_Abandon_Rate Returns

Δεν χρησιμοποιούμε Segment Label στο clustering, γιατί αυτό είναι στόχος για classification.Για clustering χρησιμοποιούμε μόνο αριθμητικές μεταβλητές που περιγράφουν τη συμπεριφορά του πελάτη

Το K-Means είναι δημοφιλές σε μια μεγάλη ποικιλία εφαρμογών λόγω της απλότητας, της αποδοτικότητας και της αποτελεσματικότητάς του. Χρησιμοποιείται ευρέως για Τμηματοποίηση Δεδομένων δηλαδή την τμηματοποίηση δεδομένων σε διακριτές ομάδες. Για παράδειγμα, οι επιχειρήσεις χρησιμοποιούν το K-Means για να ομαδοποιήσουν πελάτες με βάση τη συμπεριφορά τους, όπως τα πρότυπα αγορών ή την αλληλεπίδραση με τον ιστότοπο.Επιπλέων το K-Means μπορεί να χρησιμοποιηθεί για τη μείωση της πολυπλοκότητας των εικόνων ομαδοποιώντας παρόμοια pixel σε συστάδες, συμπιέζοντας αποτελεσματικά την εικόνα. Αυτό είναι χρήσιμο για την αποθήκευση και την επεξεργασία εικόνων. Επίσης το K-Means μπορεί να εφαρμοστεί για την ανίχνευση ανωμαλιών ή ακραίων τιμών εντοπίζοντας σημεία δεδομένων που δεν ανήκουν σε καμία από τις συστάδες. Το K-Means χρησιμοποιείται για την ομαδοποίηση παρόμοιων εγγράφων ή άρθρων. Χρησιμοποιείται συχνά σε εφαρμογές όπως συστήματα συστάσεων ή κατηγοριοποίηση ειδήσεων. Οργάνωση Μεγάλων Συνόλων Δεδομένων: Όταν πρόκειται για μεγάλα σύνολα δεδομένων, το K-Means μπορεί να βοηθήσει στην οργάνωση των δεδομένων σε μικρότερα, πιο διαχειρίσιμα τμήματα με βάση τις ομοιότητες, βελτιώνοντας την αποτελεσματικότητα της ανάλυσης δεδομένων.[1]

Στην δική μας περίπτωση θέλουμε να βρουμε ομάδες χρηστών που μοιάζουν μεταξύ τους. Άρα θέλουμε χαρακτηριστικά που περιγράφουν συμπεριφορά αγορών Frequency, Monetary, Avg Order Value, εμπλοκή με το site, Pages Viewed, Clicks , Avg Session Duration, Session Count, Wishlist Adds, Cart_Abandon_Rate , Returns . Αυτές οι τιμές έχουν νόημα για να βρεθούν clusters όπως: High-spending active users, Window shoppers , Users που μπαίνουν συχνά αλλά αγοράζουν σπάνια , Users με υψηλό abandonment .

Πως λειτουργει ο αλγοριθμος του K-means

Αρχικά, κάθε σημείο δεδομένων αντιστοιχίζεται τυχαία σε μία από τις K συστάδες. Στη συνέχεια, υπολογίζουμε το κεντροειδές (λειτουργικά το κέντρο) κάθε συστάδας και επαναπροσδιορίζουμε κάθε σημείο δεδομένων στο σύμπλεγμα με το πλησιέστερο κεντροειδές. Επαναλαμβάνουμε αυτήν τη διαδικασία μέχρι οι αντιστοιχίσεις συστάδων για κάθε σημείο δεδομένων να μην αλλάζουν πλέον. Η

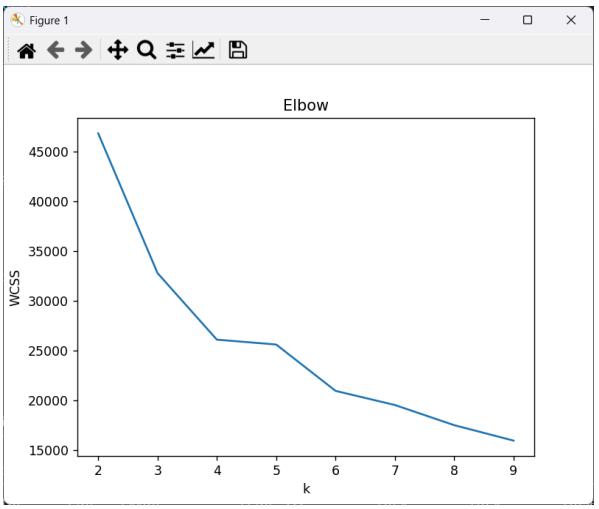
συσταδοποίηση K-means απαιτεί να επιλέξουμε το K, τον αριθμό των συστάδων στις οποίες θέλουμε να ειδοποιήσουμε τα δεδομένα. Η μέθοδος αγκώνα μας επιτρέπει να απεικονίσουμε γραφικά την αδράνεια (μια μετρική που βασίζεται στην απόσταση) και να οπτικοποιήσουμε το σημείο στο οποίο αρχίζει να μειώνεται γραμμικά. Αυτό το σημείο αναφέρεται ως "αγκώνας" και είναι μια καλή εκτίμηση για την καλύτερη τιμή για το K με βάση τα δεδομένα μας. [2]

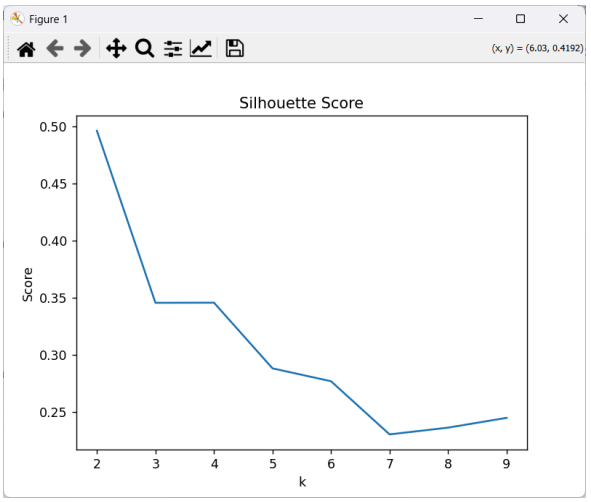
Με βαση τον παρακατω κωδικα πηραμε τα εξής αποτελέσματα

Πληρης κωδικας K-MEANS

```
# K-MEANS ALGORYTHM

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans

from sklearn.metrics import silhouette_score

import matplotlib.pyplot as plt



features = [

    "Recency","Frequency","Monetary","Avg_Order_Value",

    "Session_Count","Avg_Session_Duration","Pages_Viewed",

    "Clicks","Wishlist_Adds","Cart_Abandon_Rate","Returns"

]



X = df[features]



scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)



wcss = []

sil = []

K_range = range(2, 10)
```

```
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X_scaled)
    wcss.append(km.inertia_)
    sil.append(silhouette_score(X_scaled, labels))

plt.plot(K_range, wcss)
plt.title("Elbow")
plt.xlabel("k")
plt.ylabel("WCSS")
plt.show()

plt.plot(K_range, sil)
plt.title("Silhouette Score")
plt.xlabel("k")
plt.ylabel("Score")
plt.show()

kmeans = KMeans(n_clusters=4, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)

print(df["Cluster"].value_counts())
df.head()
```





Elbow Plot (WCSS):Το πρώτο διάγραμμα δείχνει πόσο καλά χωρίζονται οι πελάτες σε ομάδες για κάθε πιθανή τιμή του k (αριθμός clusters). Η καμπύλη πάντα κατεβαίνει, αλλά υπάρχει ένα σημείο όπου η μείωση αρχίζει να γίνεται πολύ μικρή αυτό είναι το elbow.

Silhouette Score Plot: Υπολογίζει πόσο καθαρά είναι διαχωρισμένα τα clusters μεταξύ τους. Οι υψηλότερες τιμές δείχνουν: μεγαλύτερη απόσταση ανάμεσα στις ομάδες, πιο καλά clusters, λιγότερη ανάμειξη μεταξύ των χρηστών. Το διάγραμμα δείχνει την ποιότητα κάθε επιλογής k.

Πρακτική χρήση δεδομένων του αλγορίθμου

Τα δεδομένα που προκύπτουν από τον αλγόριθμο K-Means χρησιμοποιούνται για την αναγνώριση διακριτών ομάδων πελατών με παρόμοια χαρακτηριστικά και συμπεριφορά μέσα στο ηλεκτρονικό κατάστημα. Κάθε χρήστης ταξινομείται σε ένα cluster, το οποίο αντιπροσωπεύει ένα συγκεκριμένο τύπο πελάτη, όπως υψηλής αξίας, συχνούς επισκέπτες χωρίς αγορές ή πελάτες με χαμηλή δραστηριότητα. Αυτή η ομαδοποίηση βοηθά στην καλύτερη κατανόηση των διαφορετικών μοτίβων χρήσης, στη λήψη στοχευμένων επιχειρηματικών αποφάσεων και στη δημιουργία εξατομικευμένων ενεργειών marketing. Επιπλέον, τα labels των clusters μπορούν να χρησιμοποιηθούν ως στόχος για το επόμενο στάδιο, δηλαδή την εκπαίδευση μοντέλου classification που προβλέπει το segment ενός νέου πελάτη.

Ο πίνακας των cluster profiles παρουσιάζει τους μέσους όρους όλων των συμπεριφορικών μεταβλητών για κάθε cluster και επιτρέπει την ερμηνεία των ομάδων χρηστών με βάση τη δραστηριότητα, τη δαπάνη και την αλληλεπίδρασή τους με το ηλεκτρονικό κατάστημα. Με αυτόν τον τρόπο προκύπτουν τα τελικά segments, όπως high-value users, browsers, inactive users και χρήστες με υψηλή εγκατάλειψη καλαθιού.

```
            Recency  Frequency     Monetary  Avg_Order_Value  Session_Count  ...  Pages_Viewed       Clicks  Wishlist_Adds  Cart_Abandon_Rate    Returns
Cluster                                                                      ...
0         19.496538  49.547972  3508.497527       223.584075      89.227498  ...     49.317507  1981.425321      49.814045           9.882248   4.466370
1         81.766548   8.737608   190.953695        59.596319      17.860790  ...     14.215197   354.849807       7.707628          27.604123  11.226774
2          7.511154  71.982541  6369.014791       297.498545     138.947624  ...     72.832202  2906.585597      72.400582           5.010452   2.039767
3         40.650643  22.863890  1100.084405       140.996087      46.084684  ...     32.428731   930.355506      23.186976          15.744148   6.614869

[4 rows x 11 columns]
PS C:\Users\perio\Desktop\Εξόρυξη Δεδομένων και Γνώσης>
```

Επεξήγηση κώδικα K-Means

```
features = [
    "Recency","Frequency","Monetary","Avg_Order_Value",
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Wishlist_Adds","Cart_Abandon_Rate","Returns"
]
```

Δηλώνεται η λίστα features με τα ονόματα των στηλών που θα χρησιμοποιήσουμε στο clustering.

```
X = df[features]
```

X είναι το νέο DataFrame που περιέχει μόνο τις στήλες των χαρακτηριστικών. Γιατί: έτσι δουλεύουν τα scikit-learn μοντέλα που παίρνουν ένα matrix με μόνο τα features.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

scaler = StandardScaler() δημιουργεί το αντικείμενο scaler.
X_scaled = scaler.fit_transform(X)
κάνει δύο πράγματα υπολογίζει το μέσο (mean) και την τυπική απόκλιση (std) για κάθε στήλη από τα δεδομένα X.

transform: **χρησιμοποιεί αυτά τα στατιστικά για να μετασχηματίσει κάθε τιμή σε z-score: (x - mean)/std. Το αποτέλεσμα X_scaled είναι ένας numpy πίνακας όπου κάθε**

feature έχει μέσο ~0 και std ~1. Γιατί: ο K-Means χρησιμοποιεί αποστάσεις, οπότε αν ένα feature έχει πολύ μεγαλύτερη κλίμακα, θα κυριαρχεί στην απόσταση και θα χαλάσει το clustering.

wcss = [] αρχικοποιεί λίστα για τα WCSS (Within-Cluster Sum of Squares) για κάθε k. sil = [] αρχικοποιεί λίστα για τα silhouette scores για κάθε k. K Range = range(2, 10) ορίζουμε τις τιμές k που θα δοκιμάσουμε: από 2 έως 9.

```
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X_scaled)
    wcss.append(km.inertia_)
    sil.append(silhouette_score(X_scaled, labels))
```

Για κάθε k στην K_range: km = KMeans(n_clusters=k, random_state=42) Δημιουργεί ένα KMeans αντικείμενο με k κέντρα.
random_state=42
σταθεροποιεί τα τυχαία αρχικά κέντρα ώστε να παίρνεις τα ίδια αποτελέσματα κάθε φορά.
labels = km.fit_predict(X_scaled)
Εκπαιδεύει τον K-Means πάνω σε X_scaled και επιστρέφει τα labels (πινακίδα cluster για κάθε δείγμα).
fit_predict = fit + predict. wcss.append(km.inertia_)
Το km.inertia_ είναι η τιμή WCSS (άθροισμα τετραγώνων αποστάσεων σημείων από το centroid του cluster τους). Την αποθηκεύουμε για το Elbow plot.
sil.append(silhouette_score(X_scaled, labels))
Υπολογίζει το Silhouette score για τα labels που προέκυψαν και το αποθηκεύει. Το silhouette_score υπολογίζει για κάθε δείγμα (a) μέση απόσταση προς άλλα σημεία στο ίδιο cluster και (b) τη μέση απόσταση προς σημεία στο πλησιέστερο άλλο cluster και από αυτά βγάζει ένα μέσο ποσοτικό δείκτη [-1,1].

```
plt.plot(K_range, wcss)
plt.title("Elbow")
plt.xlabel("k")
plt.ylabel("WCSS")
plt.show()

plt.plot(K_range, sil)
plt.title("Silhouette Score")
plt.xlabel("k")
plt.ylabel("Score")
plt.show()
```

Σχεδιασμός Των plots.

```
kmeans = KMeans(n_clusters=4, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

```
kmeans = KMeans n_clusters=4, random_state=42
```

Εδώ αποφασίσαμε με βάση Elbow+Silhouette να χρησιμοποιήσουμε k=4. Μπορεί να αλλάξει αν στα διαγράμματα δείχνει άλλο k.

```
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

Εκπαιδεύει τον K-Means με k=4 και δημιουργεί τη νέα στήλη Cluster στο αρχικό df με το label κάθε χρήστη (0..3). Αυτή η στήλη είναι το output labels.

```
cluster_profile = df.groupby("Cluster")[[
    "Recency","Frequency","Monetary","Avg_Order_Value",
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Wishlist_Adds","Cart_Abandon_Rate","Returns"
]].mean()

print(cluster_profile)
```

Εμφανιση πινακα clustering

CLASSIFICATION

Για την κατηγοριοποίηση επιλέξαμε τον αλγόριθμο XGBoost για το classification γιατί θεωρείται ένας από τους πιο ισχυρούς και αποδοτικούς αλγορίθμους επιβλεπόμενης μάθησης σε δομημένα δεδομένα (όπως πίνακες CSV). Χρησιμοποιεί τη λογική του gradient boosting, δηλαδή δημιουργεί πολλά αδύναμα μοντέλα (δέντρα αποφάσεων) που βελτιώνουν το ένα το άλλο, ώστε στο τέλος να προκύψει ένα πολύ ακριβές τελικό μοντέλο. Είναι επίσης ανθεκτικός σε θόρυβο και έχει υψηλές επιδόσεις χωρίς υπερβολικά πολύπλοκο κώδικα.

Τι ειναι ο XGBoost

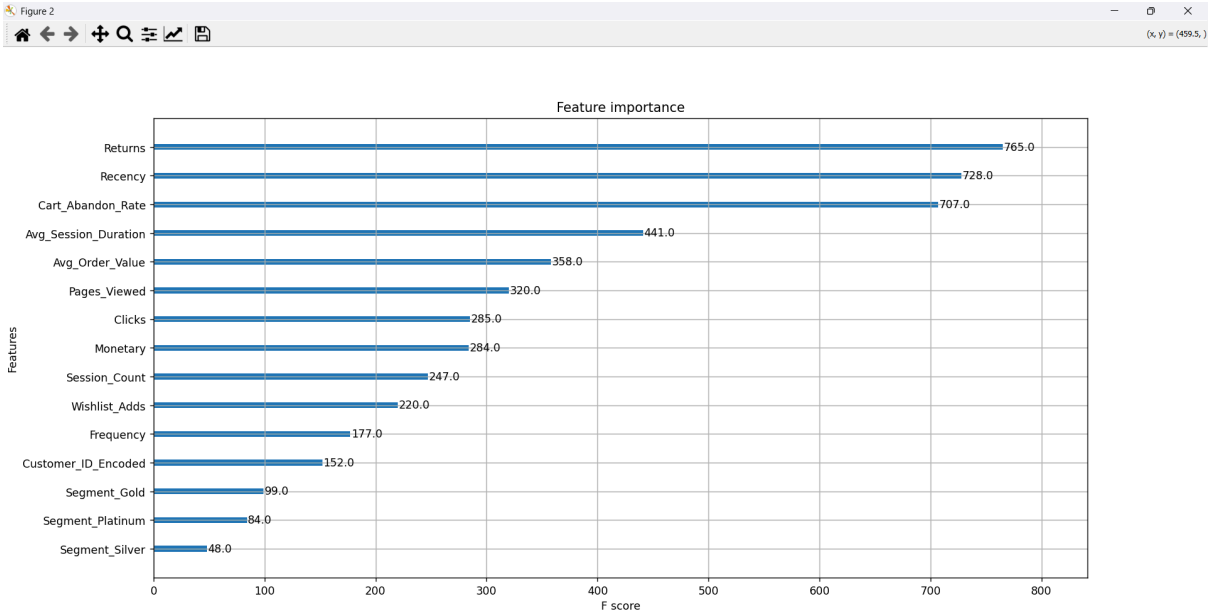
Στη μηχανική μάθηση συχνά συνδυάζουμε διαφορετικούς αλγόριθμους για να επιτύχουμε καλύτερα και βελτιστοποιημένα αποτελέσματα, κάτι που είναι γνωστό ως μέθοδος συνόλου (ensemble method) και ένας από τους διάσημους αλγόριθμους της είναι το XGBoost (Extreme boosting), το οποίο λειτουργεί δημιουργώντας ένα σύνολο δέντρων αποφάσεων διαδοχικά, όπου κάθε νέο δέντρο διορθώνει τα σφάλματα που έγιναν από το προηγούμενο. Χρησιμοποιεί προηγμένες τεχνικές βελτιστοποίησης και μεθόδους κανονικοποίησης που μειώνουν την υπερπροσαρμογή και βελτιώνουν την απόδοση του μοντέλου.Πριν προχωρήσουμε στην υλοποίηση του XG Boost, πρέπει να κατανοήσουμε τις παραμέτρους του για τη βελτιστοποίηση του μοντέλου. Ρυθμός Μάθησης (η η): Μια σημαντική μεταβλητή που τροποποιεί το πόσο συμβάλλει κάθε δέντρο στην τελική πρόβλεψη. Ενώ χρειάζονται περισσότερα δέντρα, οι μικρότερες τιμές συχνά οδηγούν σε πιο ακριβή μοντέλα. Μέγιστο βάθος: Αυτή η παράμετρος ελέγχει το βάθος κάθε δέντρου, αποφεύγοντας την υπερπροσαρμογή και είναι απαραίτητη για τον έλεγχο της πολυπλοκότητας του μοντέλου. Γάμμα : Με βάση τη μείωση της απώλειας, καθορίζει πότε ένας κόμβος στο δέντρο θα διασπαστεί. Ο αλγόριθμος γίνεται πιο συντηρητικός με υψηλότερη τιμή γάμμα, αποφεύγοντας έτσι τις διασπάσεις που δεν μειώνουν την απώλεια. Βοηθά στη διαχείριση της πολυπλοκότητας του δέντρου. Υπόδειγμα : Διαχειρίζεται το ποσοστό των δεδομένων που δειγματιζονται τυχαία για να αναπτυχθεί κάθε δέντρο, μειώνοντας έτσι τη διακύμανση και ενισχύοντας τη γενίκευση. Η ρύθμιση σε πολύ χαμηλή τιμή θα μπορούσε να οδηγήσει σε αναπροσαρμογή. Call Sample By Tree : Καθορίζει το ποσοστό των χαρακτηριστικών που θα δειγματοληπτη θούν τυχαία για την ανάπτυξη κάθε δέντρου. n estimators: Καθορίζει τον αριθμό των γύρων ενίσχυσης. άλφα (όρος κανονικοποίησης L1) και λάμδα (όρος κανονικοποίησης L2) : Ελέγχει την ισχύ της κανονικοποίησης L1 και L2 αντίστοιχα. Μια υψηλότερη τιμή έχει ως αποτέλεσμα ισχυρότερη κανονικοποίηση. min_child_weight : Επηρεάζει τη δομή δέντρου ελέγχοντας την ελάχιστη ποσότητα δεδομένων που απαιτούνται για τη δημιουργία ενός νέου κόμβου. scale_pos_weight : Χρήσιμο σε σενάρια μη ισορροπημένης κλάσης για τον έλεγχο της ισορροπίας θετικών και αρνητικών βαρών.[3]

Πλήρης κωδικας XGBoost

```
#XGBoost framework
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

X = df.drop(["Cluster", "Customer_ID"], axis=1)
y = df["Cluster"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
    subsample=0.9,
    colsample_bytree=0.9,
    objective="multi:softmax",
    num_class=4
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)


print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
from xgboost import plot_importance
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_importance(model, max_num_features=15)
plt.show()
```

Αποτελεσματα XGBoost

Η ανάλυση σημαντικότητας χαρακτηριστικών (Feature Importance) του μοντέλου XGBoost μας έδειξε ότι οι πιο σημαντικές μεταβλητές για την ταξινόμηση των πελατών είναι οι Returns, Recency και Cart_Abandon_Rate. Οι επιστροφές προϊόντων και το πόσο πρόσφατα έχει πραγματοποιηθεί μια αγορά βοηθούν σημαντικά στη διάκριση των κατηγοριών, υποδεικνύοντας ότι οι συμπεριφορές μετά την αγορά και η επαναληψιμότητα αποτελούν ισχυρούς δείκτες. Επιπλέον, δείκτες αλληλεπίδρασης με την πλατφόρμα, όπως η διάρκεια, οι προβολές σελίδων και η μέση αξία παραγγελίας, επίσης επηρεάζουν έντονα την πρόβλεψη. Αντίθετα, χαρακτηριστικά όπως οι κωδικοί πελατών ή τα ήδη υπάρχοντα segments δεν επηρεάζουν τα αποτελεσματα.



Accuracy: 0.9915

	precision	recall	f1-score	support
0	1.00	1.00	1.00	399
1	0.98	0.99	0.99	689
2	1.00	1.00	1.00	202
3	0.99	0.98	0.99	710
accuracy			0.99	2000
macro avg	0.99	0.99	0.99	2000
weighted avg	0.99	0.99	0.99	2000

Ο παραπάνω πίνακας δείχνει ότι το μοντέλο XGBoost πέτυχε πολύ υψηλό accuracy, 0.9915, δηλαδή 99% σωστές προβλέψεις. Αυτό σημαίνει ότι το μοντέλο κατάφερε να ταξινομήσει σχεδόν όλα τα δείγματα στους σωστούς clusters.
Συγκεκριμένα:
Cluster 0 Precision, recall και F1 = 1.00. Δηλαδή το μοντέλο ήταν πρακτικά τέλειο σε αυτή την κατηγορία. Cluster 1 λίγο χαμηλότερα 0.98,0.99, αλλά και πάλι πολύ υψηλά. Cluster 2 σχεδόν τέλεια απόδοση με όλα τα metrics στο 1.00. Cluster 3 λίγο χαμηλότερο recall 0.98 αλλά ακόμη πολύ υψηλό.

Επεξήγηση Κώδικα XGBoost

Χωρισμός χαρακτηριστικών και στόχου

```
X = df.drop(["Cluster", "Customer_ID"], axis=1)   Αφαιρούμε το label Cluster και το Customer_ID
y = df["Cluster"]      Η μεταβλητή στόχος είναι το cluster που βρήκε ο K-means
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42        80% εκπαίδευση και 20% test
)
```

Δημιουργία μοντέλου XGBoost

```
model = XGBClassifier(
    n_estimators=200,                    Πλήθος δέντρων
    max_depth=4,                         Μέγιστο βάθος δέντρων (πόσο πολύ μαθαίνουν)
    learning_rate=0.1,                   Ρυθμός μάθησης
    subsample=0.9,                       Ποσοστό δειγμάτων ανά δέντρο
    colsample_bytree=0.9,                Ποσοστό χαρακτηριστικών ανά δέντρο
    objective="multi:softmax",           multi-class τclassification
    num_class=4                          Συνολικός αριθμός clusters
)
```

Εκπαίδευση του μοντέλου

```
model.fit(X_train, y_train)
```

Πρόβλεψη στο test set

```
y_pred = model.predict(X_test)
```

Ακρίβεια και αναλυτικά στατιστικά

print("Accuracy:", accuracy_score(y_test, y_pred))
Πόσο σωστά προβλέπει συνολικά
print(classification_report(y_test, y_pred))
Precision, recall, f1-score ανά cluster

Εξαγωγή Κανόνων

Για την τελική εξαγωγή κανόνων επιλεξαμε τον αλγοριθμο του Apriori.Ο apriori μας υποδεικνύει κάποια χαρακτηριστικά που χρειάζονται συγκεκριμένες ομάδες πελατών.Ετσι μας βοηθούν να καταλάβουμε καλύτερα τη συμπεριφορά των πελατών και να βγάλουμε συμπεράσματα για την συμπεριφορά τους, τις συνήθειες τους και πώς μπορούμε να βελτιώσουμε τις πωλήσεις του καταστήματος.
Για να εφαρμόσουμε τον Apriori, θα χρησιμοποιήσουμε τα χαρακτηριστικά που περιγράφουν τη συμπεριφορά των πελατών, όπως τις αγορές τους, τη συχνότητα αγοράς και γενικά ό,τι μπορεί να μετατραπεί σε κανόνες. Αυτά τα δεδομένα μας δείχνουν χαρακτηριστικά εμφανίζονται συχνά μαζί στους ίδιους πελάτες. Έτσι μπορούμε να βρούμε συνδυασμούς συνηθειών που πάνε μαζί και να τους χρησιμοποιήσουμε για προτάσεις, στοχευμένες προσφορές και καλύτερη κατανόηση της πελατειακής συμπεριφοράς.

Τα δεδομένα που θα χρησιμοποιήσουμε απο το dataset για τον αλγόριθμο ειναι τα εξής
Campaign_Response
Wishlist Adds
Returns
Cart_Abandon_Rate
Segment Label

Πώς λειτουργεί ο αλγόριθμος Apriori

Ο αλγόριθμος Apriori λειτουργεί μέσω μιας συστηματικής διαδικασίας που περιλαμβάνει πολλά βασικά βήματα: 1. Προσδιορισμός Συχνών Συνόλων Στοιχείων
Ο αλγόριθμος Apriori ξεκινά εξετάζοντας όλα τα δεδομένα για να μετρήσει πόσες φορές εμφανίζεται κάθε μεμονωμένο στοιχείο. Αυτά τα μεμονωμένα στοιχεία ονομάζονται Σύνολα 1-Στοιχείου. Στη συνέχεια, χρησιμοποιεί έναν κανόνα που ονομάζεται ελάχιστη υποστήριξη. Αυτός είναι ένας αριθμός που μας λέει πόσο συχνά

ένα στοιχείο ή μια ομάδα στοιχείων πρέπει να εμφανίζεται ως σημαντικό. Εάν ένα στοιχείο εμφανίζεται αρκετά συχνά, που σημαίνει ότι ο αριθμός του είναι πάνω από αυτήν την ελάχιστη υποστήριξη, ονομάζεται συχνό σύνολο στοιχείων. 2. Δημιουργία πιθανής ομάδας στοιχείων Αφού εντοπίσει τα μεμονωμένα στοιχεία που εμφανίζονται αρκετά συχνά (συχνές ομάδες 1 στοιχείου), ο αλγόριθμος τα συνδυάζει για να δημιουργήσει ζεύγη στοιχείων (ομάδες 2 στοιχείων). Στη συνέχεια, ελέγχει ποια ζεύγη είναι συχνά, βλέποντας αν εμφανίζονται αρκετές φορές στα δεδομένα. Αυτή η διαδικασία συνεχίζεται βήμα προς βήμα, δημιουργώντας ομάδες των 3 στοιχείων, στη συνέχεια των 4 στοιχείων και ούτω καθεξής. Ο αλγόριθμος σταματά όταν δεν μπορεί να βρει μεγαλύτερες ομάδες που να εμφανίζονται αρκετά συχνά. 3. Αφαίρεση ομάδων σπάνιων στοιχείων Ο αλγόριθμος Apriori χρησιμοποιεί έναν χρήσιμο κανόνα για εξοικονόμηση χρόνου. Αυτός ο κανόνας λέει: εάν μια ομάδα στοιχείων δεν εμφανίζεται αρκετά συχνά, τότε οποιαδήποτε μεγαλύτερη ομάδα που περιλαμβάνει αυτά τα στοιχεία δεν θα εμφανίζεται επίσης συχνά. Εξαιτίας αυτού, ο αλγόριθμος δεν ελέγχει αυτές τις μεγαλύτερες ομάδες. Με αυτόν τον τρόπο αποφεύγεται η σπατάλη χρόνου εξετάζοντας ομάδες που δεν θα είναι σημαντικές, επιταχύνοντας έτσι ολόκληρη τη διαδικασία. 4. Δημιουργία κανόνων συσχέτισης Ο αλγόριθμος δημιουργεί κανόνες για να δείξει πώς σχετίζονται τα στοιχεία. Ελέγχει αυτούς τους κανόνες χρησιμοποιώντας υποστήριξη, αυτοπεποίθηση και ενθάρρυνση για να βρει τους ισχυρότερους. Βασικές μετρήσεις του αλγορίθμου Apriori Υποστήριξη : Αυτή η μέτρηση μετρά πόσο συχνά εμφανίζεται ένα στοιχείο στο σύνολο δεδομένων σε σχέση με τον συνολικό αριθμό συναλλαγών. Μια υψηλότερη υποστήριξη υποδηλώνει μια πιο σημαντική παρουσία του συνόλου στοιχείων στο σύνολο δεδομένων. Η υποστήριξη μας λέει πόσο συχνά εμφανίζεται ένα συγκεκριμένο στοιχείο ή συνδυασμός στοιχείων σε όλες τις συναλλαγές, όπως το ψωμί που αγοράζεται στο 20% όλων των συναλλαγών. Εμπιστοσύνη : Η εμπιστοσύνη αξιολογεί την πιθανότητα αγοράς ενός προϊόντος Y όταν αγοράζεται το προϊόν X. Παρέχει πληροφορίες για την ισχύ της συσχέτισης μεταξύ δύο προϊόντων. Η εμπιστοσύνη μας λέει πόσο συχνά τα προϊόντα συνδυάζονται, π.χ. αν αγοράζεις ψωμί, το βούτυρο αγοράζεται στο 75% των περιπτώσεων. Αύξηση : Η αύξηση αξιολογεί πόσο πιο πιθανό είναι δύο είδη να αγοραστούν μαζί σε σύγκριση με την αγορά τους ξεχωριστά. Μια αύξηση μεγαλύτερη από 1 υποδηλώνει ισχυρή θετική συσχέτιση. Η αύξηση δείχνει πόσο ισχυρή είναι η σύνδεση μεταξύ των ειδών. Όπως το ψωμί και το βούτυρο είναι πολύ πιο πιθανό να αγοραστούν μαζί παρά τυχαία.[4]

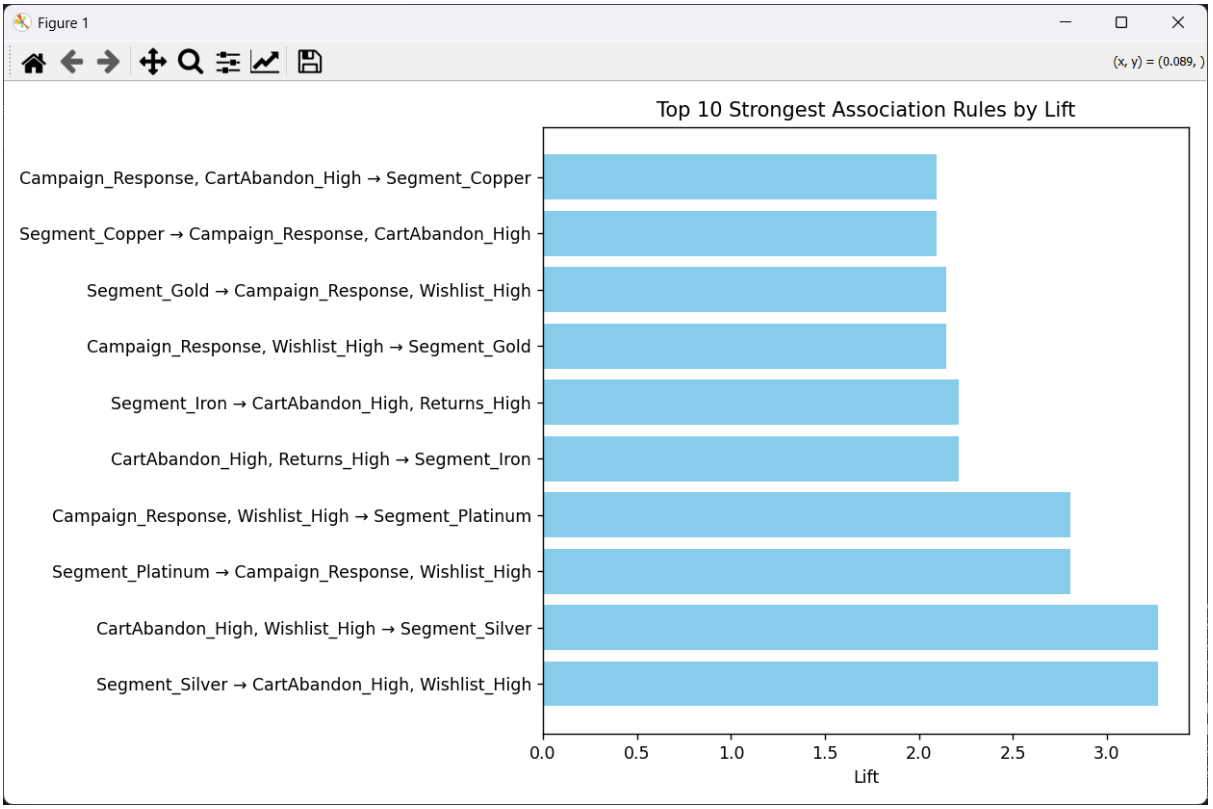
Πλήρης Κώδικας Apriori

```
from mlxtend.frequent_patterns import apriori, association_rules

segment_ohe = df.filter(like="Segment_")

apriori_features = list(segment_ohe.columns) + [
```

```
    "Wishlist_Adds",
    "Cart_Abandon_Rate",
    "Campaign_Response",
    "Returns"
]


df_ap = df[apriori_features].copy()

segment_ohe = df.filter(like="Segment_")


df_ap["Campaign_Response"] = df_ap["Campaign_Response"].apply(lambda x: 1 if x > 0 else 0)


df_ap["Wishlist_High"] = df["Wishlist_Adds"].apply(lambda x: 1 if x >
df["Wishlist_Adds"].median() else 0)


df_ap["CartAbandon_High"] = df["Cart_Abandon_Rate"].apply(lambda x: 1 if x >
df["Cart_Abandon_Rate"].median() else 0)


df_ap["Returns_High"] = df["Returns"].apply(lambda x: 1 if x > df["Returns"].median() else
0)


basket = pd.concat([
    segment_ohe,
    df_ap[["Campaign_Response", "Wishlist_High", "CartAbandon_High", "Returns_High"]]
], axis=1)

basket = basket.astype(bool).astype(int)

frequent_items = apriori(basket, min_support=0.05, use_colnames=True)

rules = association_rules(frequent_items, metric="lift", min_threshold=1.1)

print("Frequent Itemsets:")
print(frequent_items.head())

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())


strong_rules = rules.sort_values("lift", ascending=False).head(10)
print("\nTop 10 strongest rules:")
print(strong_rules)

plt.figure(figsize=(10,6))
plt.barh(range(len(strong_rules)), strong_rules["lift"], color='skyblue')
plt.yticks(range(len(strong_rules)), [", ".join(list(a)) + " → " + ", ".join(list(c))
```

```
                                    for a, c in zip(strong_rules["antecedents"],
strong_rules["consequents"])])
plt.xlabel("Lift")
plt.title("Top 10 Strongest Association Rules by Lift")
plt.tight_layout()
plt.show()
```

Βάση του παραπάνω κώδικα πήραμε τα εξης αποτελέσματα



Το γράφημα δείχνει τους 10 πιο δυνατούς κανόνες συσχέτισης και ουσιαστικά μας λέει ποιοι πελάτες με συγκεκριμένες συμπεριφορές μοιάζουν περισσότερο μεταξύ τους. Για παράδειγμα βλέπουμε ότι οι πελάτες του segment Silver τείνουν να έχουν υψηλό cart abandon και υψηλά wishlist adds, ενώ το segment Platinum συνδέεται έντονα με υψηλό wishlist και υψηλή ανταπόκριση σε καμπάνιες. Το segment Gold εμφανίζει επίσης ισχυρή σχέση με campaign response και υψηλό wishlist. Αντίστοιχα, το segment Iron και το segment Copper συνδέονται συχνά με υψηλό cart abandon, returns και campaign response. Με απλά λόγια το γράφημα μας δείχνει ποιοι τύποι πελατών έχουν κοινές αγοραστικές και online συμπεριφορές και έτσι μπορούμε να καταλάβουμε καλύτερα τις ανάγκες τους.

```
C:\Users\perio\AppData\Local\Programs\Python\Python312\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:161: DeprecationWarning: DataFrames with
non-bool types result in worse computationalperformance and their support might be discontinued in the future.Please use a DataFrame with bool type
  warnings.warn(
Frequent Itemsets:
   support            itemsets
0   0.2444    (Segment_Copper)
1   0.2022      (Segment_Gold)
2   0.1448      (Segment_Iron)
3   0.1031  (Segment_Platinum)
4   0.3055    (Segment_Silver)

Association Rules:
           antecedents         consequents  support  confidence      lift
0     (Segment_Copper)  (CartAbandon_High)   0.2294    0.938625  1.877250
1   (CartAbandon_High)    (Segment_Copper)   0.2294    0.458800  1.877250
2       (Returns_High)    (Segment_Copper)   0.1604    0.331679  1.357116
3     (Segment_Copper)      (Returns_High)   0.1604    0.656301  1.357116
4  (Campaign_Response)      (Segment_Gold)   0.1231    0.295416  1.461011

Top 10 strongest rules:
                              antecedents                           consequents  antecedent support  ...   jaccard  certainty  kulczynski
63                      (Segment_Silver)      (Wishlist_High, CartAbandon_High)              0.3055  ...  0.250082   0.188049    0.625041
60      (Wishlist_High, CartAbandon_High)                      (Segment_Silver)              0.0764  ...  0.250082   1.000000    0.625041
53                    (Segment_Platinum)     (Wishlist_High, Campaign_Response)              0.1031  ...  0.269534   0.715682    0.542903
48     (Wishlist_High, Campaign_Response)                    (Segment_Platinum)              0.2836  ...  0.269534   0.207818    0.542903
43       (Returns_High, CartAbandon_High)                      (Segment_Iron)              0.3184  ...  0.282037   0.204908    0.511883
46                       (Segment_Iron)       (Returns_High, CartAbandon_High)              0.1448  ...  0.282037   0.565331    0.511883
37                       (Segment_Gold)     (Wishlist_High, Campaign_Response)              0.2022  ...  0.339399   0.453941    0.521433
32     (Wishlist_High, Campaign_Response)                      (Segment_Gold)              0.2836  ...  0.339399   0.290627    0.521433
24                     (Segment_Copper)  (Campaign_Response, CartAbandon_High)              0.2444  ...  0.221253   0.169200    0.396310
23  (Campaign_Response, CartAbandon_High)                    (Segment_Copper)              0.1337  ...  0.221253   0.354607    0.396310

[10 rows x 14 columns]
PS C:\Users\perio\Desktop\Εξόρυξη Δεδομένων και Γνώσης>
```

Η έξοδος δείχνει τι έμαθε ο Apriori από το dataset. Πρώτα εμφανίζονται τα πιο συχνά segments, όπου το Segment_Silver και το Segment_Copper έχουν τη μεγαλύτερη παρουσία στους πελάτες. Στη συνέχεια παρουσιάζονται οι πρώτοι κανόνες συσχέτισης, όπου βλέπουμε ότι όταν κάποιος ανήκει στο Segment Copper υπάρχει αυξημένη πιθανότητα να έχει υψηλό Cart Abandon Rate, κάτι που φαίνεται και από την υψηλή confidence. Στον πίνακα με τους 10 ισχυρότερους κανόνες βλέπουμε συνδυασμούς χαρακτηριστικών που συνδέονται έντονα μεταξύ τους, όπως το ότι χρήστες του Segment Silver τείνουν να έχουν και υψηλό Wishlist και υψηλό Cart Abandon Rate, ή ότι το Segment Platinum συνδέεται συχνά με υψηλά wishlist.

Επεξήγηση κώδικα Apriori

Παίρνουμε μόνο τις one-hot encoded στήλες των segments

```
segment_ohe = df.filter(like="Segment_")
```

Ορίζουμε τις στήλες που θα χρησιμοποιήσουμε για το Apriori

```
apriori_features = list(segment_ohe.columns) + [
    "Wishlist_Adds",
    "Cart_Abandon_Rate",
    "Campaign_Response",
    "Returns"
]
```

Φτιάχνουμε νέο DataFrame μόνο με αυτές τις στήλες

```
df_ap = df[apriori_features].copy()
```

Ξαναπαίρνουμε segment_ohe

```
segment_ohe = df.filter(like="Segment_")
```

Κάνουμε δυαδική την στήλη Campaign_Response 0 ή 1

```
df_ap["Campaign_Response"] = df_ap["Campaign_Response"].apply(lambda x: 1 if x > 0
else 0)
```

Φτιάχνουμε δυαδική μεταβλητή για Wishlist με βάση τη διάμεσο

```
df_ap["Wishlist_High"] = df["Wishlist_Adds"].apply(lambda x: 1 if x >
df["Wishlist_Adds"].median() else 0)
```

Το ίδιο και για Cart Abandon Rate

```
df_ap["CartAbandon_High"] = df["Cart_Abandon_Rate"].apply(lambda x: 1 if x >
df["Cart_Abandon_Rate"].median() else 0)
```

Και για τα Returns

```
df_ap["Returns_High"] = df["Returns"].apply(lambda x: 1 if x > df["Returns"].median() else 0)
```

Ενώνουμε όλα τα δυαδικά features σε ένα καλάθι για τον Apriori

```
basket = pd.concat([
    segment_ohe,
    df_ap[["Campaign_Response", "Wishlist_High", "CartAbandon_High", "Returns_High"]]
], axis=1)
```

Μετατρέπουμε τα πάντα σε 0/1 (το Apriori τα χρειάζεται σε boolean μορφή)

```
basket = basket.astype(bool).astype(int)
```

Βρίσκουμε τα συχνά σύνολα αντικειμένων με min_support=0.05

```
frequent_items = apriori(basket, min_support=0.05, use_colnames=True)
```

Εξάγουμε τους κανόνες συσχέτισης με βάση το lift

```
rules = association_rules(frequent_items, metric="lift", min_threshold=1.1)

print("Frequent Itemsets:")
print(frequent_items.head())

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())
```

Παίρνουμε τους 10 ισχυρότερους κανόνες με βάση το lift

```
strong_rules = rules.sort_values("lift", ascending=False).head(10)
print("\nTop 10 strongest rules:")
print(strong_rules)
```

Κάνουμε visualization των top 10 κανόνων

```
plt.figure(figsize=(10,6))
plt.barh(range(len(strong_rules)), strong_rules["lift"], color='skyblue')
```

Βάζουμε στον άξονα Y το κείμενο Antecedents Consequents

```
plt.yticks(
    range(len(strong_rules)),
    [", ".join(list(a)) + " → " + ", ".join(list(c))
     for a, c in zip(strong_rules["antecedents"], strong_rules["consequents"])]
)

plt.xlabel("Lift")
plt.title("Top 10 Strongest Association Rules by Lift")
plt.tight_layout()
plt.show()
```

Τελικός Κώδικας

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder


# Φόρτωμα dataset
df = pd.read_csv("C:/Users/perio/Desktop/Εξόρυξη Δεδομένων και Γνώσης/ecommerecenice.csv")

print("Initial shape:", df.shape)
print(df.head())
print(df.info())


#  Missing values

print("\nMissing values per column:")
print(df.isna().sum())

# Πεδία που ΔΕΝ γίνεται να έχουν missing = αφαίρεση γραμμών
df = df.dropna(subset=["Customer_ID"])

# Αριθμητικά  συμπλήρωση με διάμεσο
numeric_cols = [
    "Recency","Frequency","Monetary","Avg_Order_Value",
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Campaign_Response","Wishlist_Adds",
    "Cart_Abandon_Rate","Returns"
]

for col in numeric_cols:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].median())

# Κατηγορικά Unknown
```

```python
df["Segment_Label"] = df["Segment_Label"].fillna("Unknown")


#  Noise handling

# Αρνητικές τιμές δεν γίνεται να υπάρχουν εδώ
for col in numeric_cols:
    df = df[df[col] >= 0]

# Session duration πρέπει να είναι λογική
df = df[df["Avg_Session_Duration"] < 300]


#  Outlier handling (IQR capping)

def cap_outliers(series):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    low = Q1 - 1.5 * IQR
    high = Q3 + 1.5 * IQR
    return series.clip(lower=low, upper=high)

for col in numeric_cols:
    df[col] = cap_outliers(df[col])


#  Inconsistencies / Standardization

# Καθαρισμός Customer_ID
df["Customer_ID"] = df["Customer_ID"].str.strip()

# Standardization για Segment_Label
df["Segment_Label"] = df["Segment_Label"].str.strip().str.title()

# Βεβαίωση ότι έχει μόνο έγκυρες κατηγορίες
valid_segments = ["Bronze", "Silver", "Gold", "Platinum", "Iron", "Copper"]
df["Segment_Label"] = df["Segment_Label"].apply(
    lambda x: x if x in valid_segments else "Unknown"
)

# ΚΩΔΙΚΟΠΟΙΗΣΗ USER ID
# Κάθε πελάτης παίρνει έναν μοναδικό αριθμό

user_encoder = LabelEncoder()
df["Customer_ID_Encoded"] = user_encoder.fit_transform(df["Customer_ID"])


#  ΚΩΔΙΚΟΠΟΙΗΣΗ SEGMENT LABEL (One-Hot Encoding)
# Δημιουργεί στήλες Segment_Gold, Segment_Silver, Segment_Iron

df = pd.get_dummies(df,
                    columns=["Segment_Label"],
```

```
                    prefix="Segment",
                    drop_first=False)


print(df.filter(like="Segment").head())
print(df[["Customer_ID", "Customer_ID_Encoded"]].head())

#  Remove duplicates

before = df.shape[0]
df = df.drop_duplicates(subset=["Customer_ID"])
after = df.shape[0]
print(f"\nRemoved duplicates: {before - after}")

#  Data validation checks

# Avg_Order_Value <= Monetary
invalid = df[df["Avg_Order_Value"] > df["Monetary"]]
print("\nEntries with impossible order values:", invalid.shape[0])

# Frequency must be >= Session_Count? (λογικό: orders <= sessions)
invalid2 = df[df["Frequency"] > df["Session_Count"]]
print("Entries with Frequency > Sessions:", invalid2.shape[0])


#  Final standardization

# Όλες οι αριθμητικές στήλες σε float
df[numeric_cols] = df[numeric_cols].astype(float)

# Αποθήκευση καθαρού dataset
df.to_csv("clean_preprocessed_ecommerce.csv", index=False)

print("\n Saved as clean_preprocessed_ecommerce.csv")
print(df.head())



# K-MEANS ALGORYTHM

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

features = [
    "Recency","Frequency","Monetary","Avg_Order_Value",
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Wishlist_Adds","Cart_Abandon_Rate","Returns"
]

X = df[features]
```

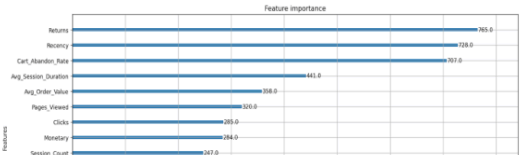
```python
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

wcss = []
sil = []
K_range = range(2, 10)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    labels = km.fit_predict(X_scaled)
    wcss.append(km.inertia_)
    sil.append(silhouette_score(X_scaled, labels))

plt.plot(K_range, wcss)
plt.title("Elbow")
plt.xlabel("k")
plt.ylabel("WCSS")
plt.show()

plt.plot(K_range, sil)
plt.title("Silhouette Score")
plt.xlabel("k")
plt.ylabel("Score")
plt.show()

kmeans = KMeans(n_clusters=4, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)

print(df["Cluster"].value_counts())
df.head()

cluster_profile = df.groupby("Cluster")[[
    "Recency","Frequency","Monetary","Avg_Order_Value",
    "Session_Count","Avg_Session_Duration","Pages_Viewed",
    "Clicks","Wishlist_Adds","Cart_Abandon_Rate","Returns"
]].mean()

print(cluster_profile)

#XGBoost Algorythm
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

X = df.drop(["Cluster", "Customer_ID"], axis=1)
y = df["Cluster"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = XGBClassifier(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.1,
```

```
    subsample=0.9,
    colsample_bytree=0.9,
    objective="multi:softmax",
    num_class=4
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)


print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

from xgboost import plot_importance
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_importance(model, max_num_features=15)
plt.show()

# Apriori Algorythm
from mlxtend.frequent_patterns import apriori, association_rules

segment_ohe = df.filter(like="Segment_")

apriori_features = list(segment_ohe.columns) + [
    "Wishlist_Adds",
    "Cart_Abandon_Rate",
    "Campaign_Response",
    "Returns"
]


df_ap = df[apriori_features].copy()

segment_ohe = df.filter(like="Segment_")


df_ap["Campaign_Response"] = df_ap["Campaign_Response"].apply(lambda x: 1 if x > 0 else 0)


df_ap["Wishlist_High"] = df["Wishlist_Adds"].apply(lambda x: 1 if x >
df["Wishlist_Adds"].median() else 0)


df_ap["CartAbandon_High"] = df["Cart_Abandon_Rate"].apply(lambda x: 1 if x >
df["Cart_Abandon_Rate"].median() else 0)


df_ap["Returns_High"] = df["Returns"].apply(lambda x: 1 if x > df["Returns"].median() else
0)
```

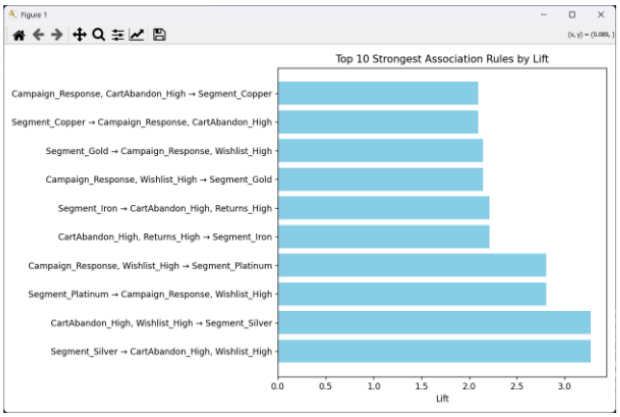
```python
basket = pd.concat([
    segment_ohe,
    df_ap[["Campaign_Response", "Wishlist_High", "CartAbandon_High", "Returns_High"]]
], axis=1)

basket = basket.astype(bool).astype(int)

frequent_items = apriori(basket, min_support=0.05, use_colnames=True)

rules = association_rules(frequent_items, metric="lift", min_threshold=1.1)

print("Frequent Itemsets:")
print(frequent_items.head())

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']].head())


strong_rules = rules.sort_values("lift", ascending=False).head(10)
print("\nTop 10 strongest rules:")
print(strong_rules)
```

Επιχειρησιακές Αποφάσεις Βασισμένες στα Αποτελέσματα της Ανάλυσης

Τα αποτελέσματα του XGBoost δείχνουν ότι οι μεταβλητές **Recency**, **Returns** και **Cart_Abandon_Rate** αποτελούν τους πιο σημαντικούς παράγοντες που επηρεάζουν τη συμπεριφορά των χρηστών και τη δημιουργία των clusters. Αυτό κατευθύνει την επιχείρηση σε στοχευμένες ενέργειες για χρήστες που έχουν μεγάλο διάστημα να



επισκεφθούν το κατάστημα, καθώς και σε βελτίωση των περιγραφών προϊόντων και των πολιτικών επιστροφών για τη μείωση του κόστους από συχνές επιστροφές. Η υψηλή σημασία της εγκατάλειψης καλαθιού δείχνει την ανάγκη για απλούστερη διαδικασία checkout, διαφάνεια στο κόστος αποστολής και αποστολή email υπενθύμισης για μεγάλα καλάθια. Ταυτόχρονα, τα clicks,η διάρκεια, οι προβολές σελίδων και οι προσθήκες wishlist δείχνουν ότι η επιχείρηση μπορεί να ενισχύσει την απόδοση της με βελτιωμένη εμπειρία χρήστη, εξατομικευμένες προτάσεις προϊόντων σε όλους τουσ χρήστες και διαφημιστικές καμπάνιες σε χρήστες με υψηλό Avg_Order_Value ή Monetary.



Η ανάλυση Association Rules ενισχύει περαιτέρω αυτές τις στρατηγικές, αποκαλύπτοντας χαρακτηριστικα συμπεριφοράς χρηστών. Για παράδειγμα, κανόνες όπως **CartAbandon_High, Wishlist_High Segment_Silver** ή **Campaign_Response, Wishlist_High Segment_Platinum,** υποδεικνύουν ότι χρήστες που δείχνουν έντονο ενδιαφέρον π.χ. προσθήκες στη wishlist αλλά διστάζουν να ολοκληρώσουν αγορά καταλήγουν συχνά σε συγκεκριμένες ομάδες πελατών. Αντίστοιχα, ο κανόνας **CartAbandon_High, Returns_High , Segment_Iron** φανερώνει μια ομάδα χρηστών υψηλού ρίσκου, για τους οποίους απαιτούνται αυστηρότερες πολιτικές αλλαγών. Τέτοιες συσχετίσεις επιτρέπουν στην επιχείρηση να εφαρμόσει διαφοροποιημένες στρατηγικές ανά ομάδα χρηστών, βελτιώνοντας τα έσοδα, αυξάνοντας τις θετικές κριτικές και μειώνοντας τα περιττά κόστη από επιστροφές και χαμένες πωλήσεις.

Πηγές

[1] https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/
[2]https://www.w3schools.com/python/python_ml_k-means.asp
[3] https://www.geeksforgeeks.org/machine-learning/implementation-of-xgboost-extreme-gradient-boosting/
[4] https://www.geeksforgeeks.org/machine-learning/apriori-algorithm/
[5]Dataset from: https://www.kaggle.com/datasets/ziya07/e-commerce-user-behavior-and-transaction-dataset